IY 453 Course work stage 3

Module Code	IY453
Group	B
Module Title	IY453 Software Design and Implementation
Assessment Type	Cousrsework satge 3
Module Tutor Name	Jonathan Shore
Student ID Number	P469808
Date of Submission	15.03.2026

- [x] *I confirm that this assignment is my own work. Where I have referred to academic sources, I have provided in-text citations and included the sources in the final reference list.*

- [x] *Where I have used AI, I have cited and referenced appropriately.

Contents

- Introduction
- Analysis and Design
  - Program Specification
  - IPO Table
  - Algorithms
  - Flowcharts
  - Class Diagram
- References

Introduction:

The aim of this program is to create an interactive text-based “choose your own adventure” game in C++ that delivers a structured story with branching paths based on the player’s choices. The player progresses through scenes by selecting between two options, with consequences that may lead to success, new routes, item collection, or game over. To enhance gameplay and replayability, the program includes player name personalisation, inventory management with meaningful item effects, puzzle and combat encounters, a scoring system, and save/load

features, while applying strong input validation to prevent crashes and handle invalid inputs reliably.

Analysis and Design

Game flow

- Display an introduction/start menu (New Game / Load Game / Exit).
- Start a new game by asking for the player's name.
- Present scenes that always provide two options (Option 1 / Option 2).
- Move to the next scene based on the selected option.
- Detect and display Game Over and allow the player to restart.
- Detect and display a win ending and show the final score.

Input validation

- Validate all user choices so only valid options are accepted (e.g., 1–2).
- Validate puzzle answers (not empty, correct format if numeric).
- Validate save/load actions (file exists, file format is valid).
- Print clear error messages and re-prompt the user when input is invalid.

Inventory system

- Allow the player to collect items during the game.
- Allow the player to view inventory at appropriate times.
- Allow the player to use items (where relevant).
- Items must affect gameplay by:
    - modifying health / attack / defense, and/or
    - unlocking routes (e.g., "key required" checks).

Player stats

- Store and update player stats such as:

    - Health (HP)
    - Attack (ATK)
    - Defense (DEF)
- Ensure stats influence gameplay outcomes (especially combat).

Puzzle scenarios

- Include at least 2 puzzle scenarios (one riddle/anagram + one math puzzle) to ensure variety.
- Check the user's answer and apply consequences:
    - reward (score increase, item, route unlock), or
    - penalty (HP loss, lower score, harder route).

Combat scenarios

- Include at least one combat scenario influenced by player stats and inventory.
- Combat outcome must be determined by a fair rule (stats + randomness).
- Combat must result in either:
    - win (score reward, possible item), or
    - loss (HP reduction or Game Over).

Scoring system

- Track score throughout the game.
- Increase score for completing puzzles, winning combat, or making good decisions.
- Display score at key moments and show a final score at the end.

Save and load

- Provide the ability to save and resume the game later.
- Saved data must include:
    - player name

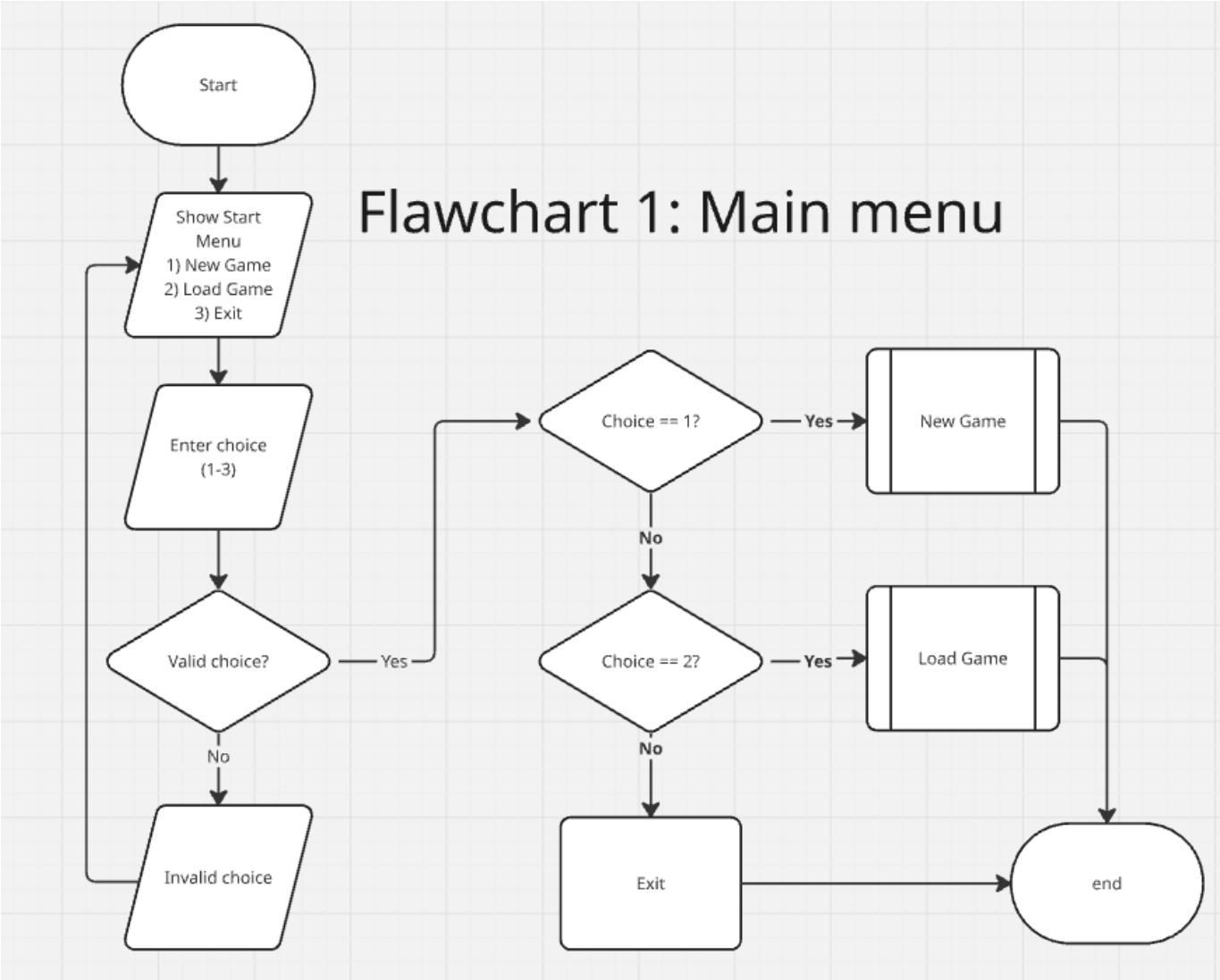
    - current scene ID (progress point)
    - player stats (HP/ATK/DEF)
    - score
    - inventory contents
- Loading must restore the game to the exact saved state.

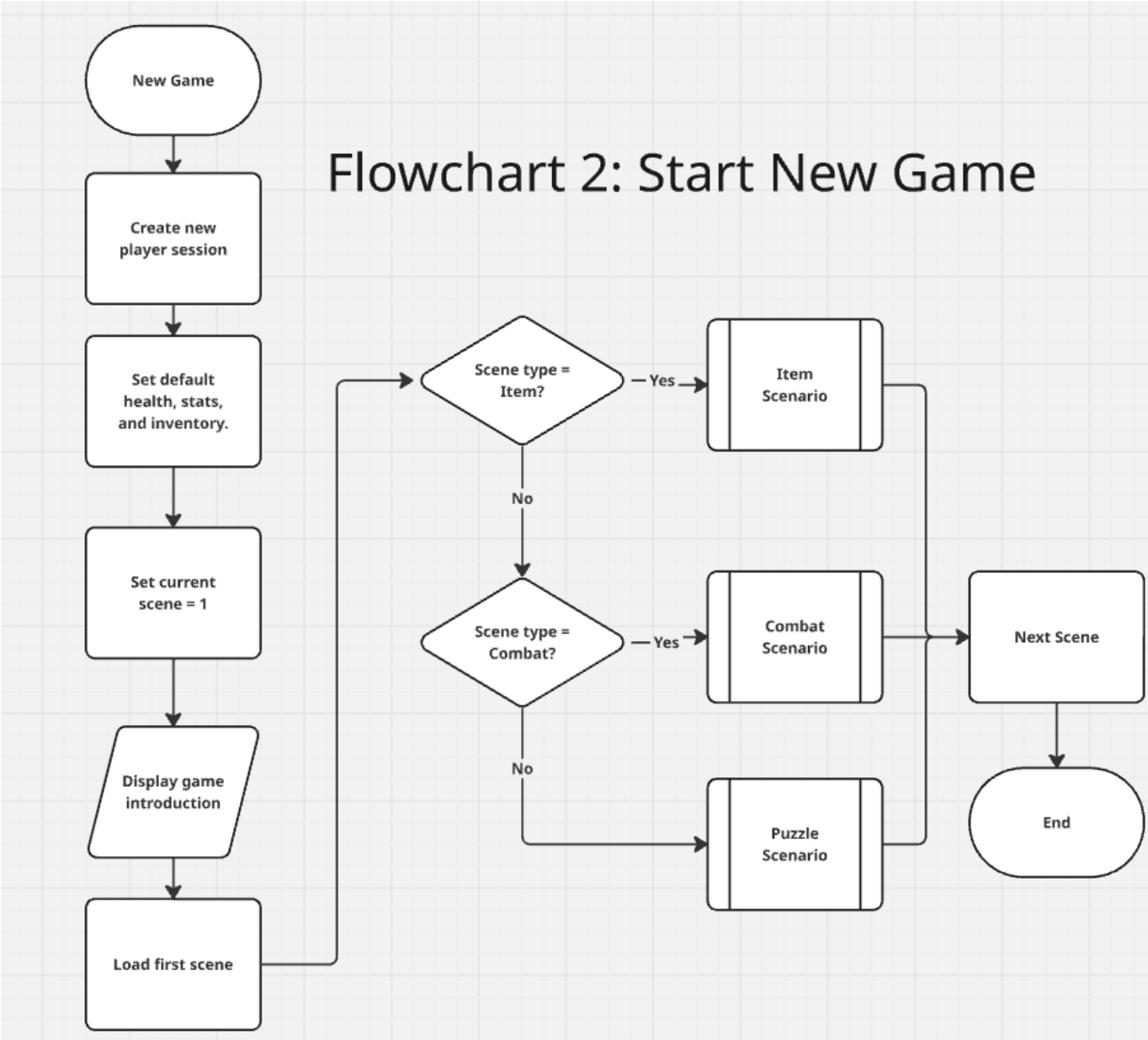
IPO Table

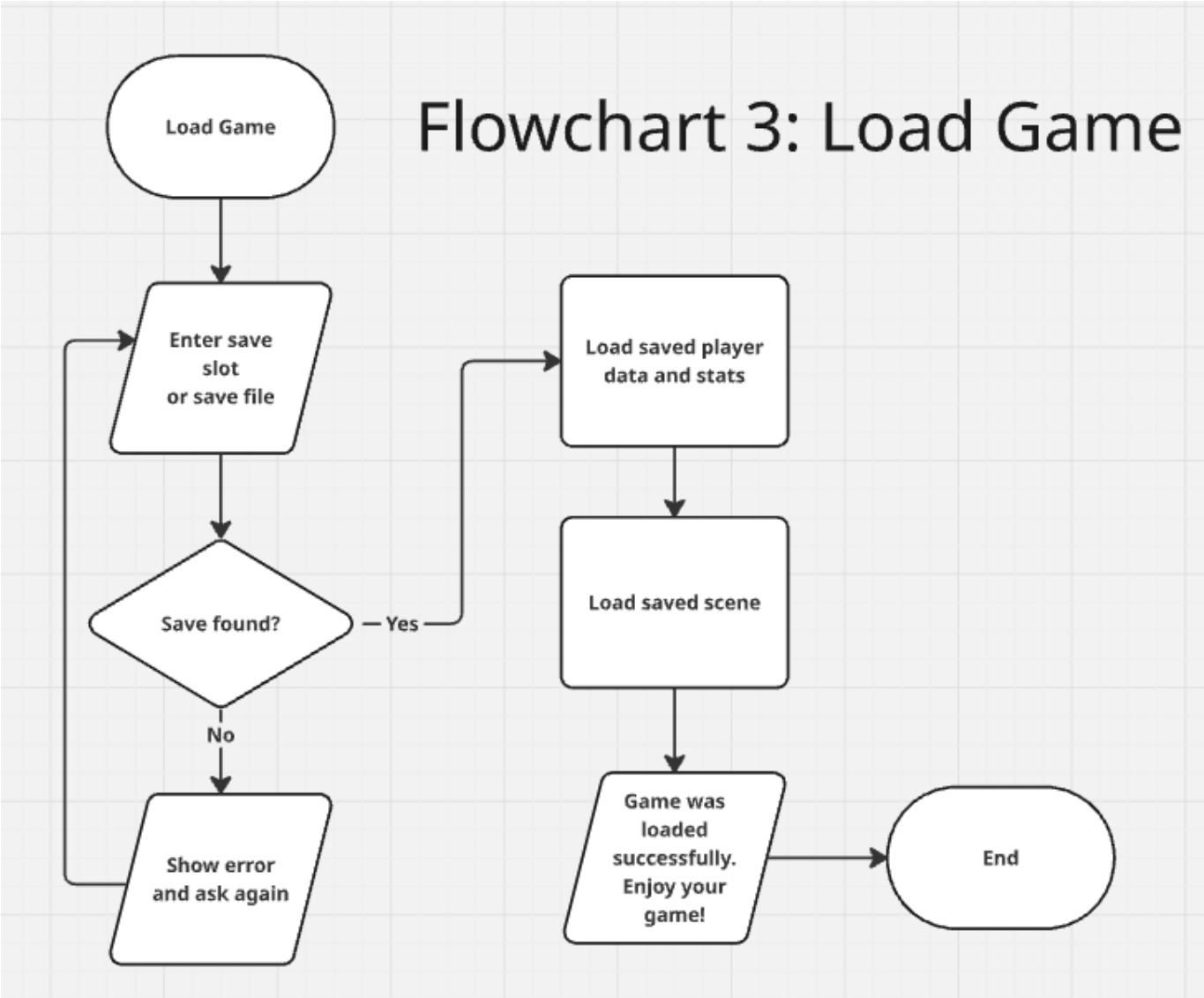
Feature	Inputs	Processing	Outputs
Show start menu & get choice	User menu choice (1–3)	Display start menu, read input, validate range; if invalid show error and re-prompt.	A valid menu option selected (1–3).
Choice 1: Start new game	User menu choice = 1, player name (string)	Ask for player name, validate not empty, initialise starting stats/score/inventory, set starting scene.	Welcome message using player name + first scene displayed.
Choice 2: Load game	User menu choice = 2, filename/slot (string)	Validate file exists, read saved state, validate format, restore player stats/score/inventory and current scene.	Load success message + resumed scene OR load error message.
Choice 3: Exit game	User menu choice = 3	End the program cleanly.	Goodbye message.
Play a scene (binary choice)	Scene choice (1 or 2)	Display scene narrative + 2 options, validate input, apply consequences, move to next scene.	Next scene displayed + consequence message (and score/stats updated if relevant).
Puzzle scenario	Puzzle answer (string/number)	Ask puzzle question, validate input, check answer, apply reward/penalty and decide next route.	Correct/incorrect feedback + updated score/stats + next scene.
Combat scenario	Combat choice (e.g., 1 = Fight, 2 = Run)	Validate input, calculate combat using stats + randomness, apply HP/score changes, decide next route or Game Over.	Combat outcome message + updated HP/score + next scene OR Game Over.
View inventory	Inventory request (Y/N)	If requested, display inventory list and return to gameplay.	Inventory list OR “inventory empty” message.

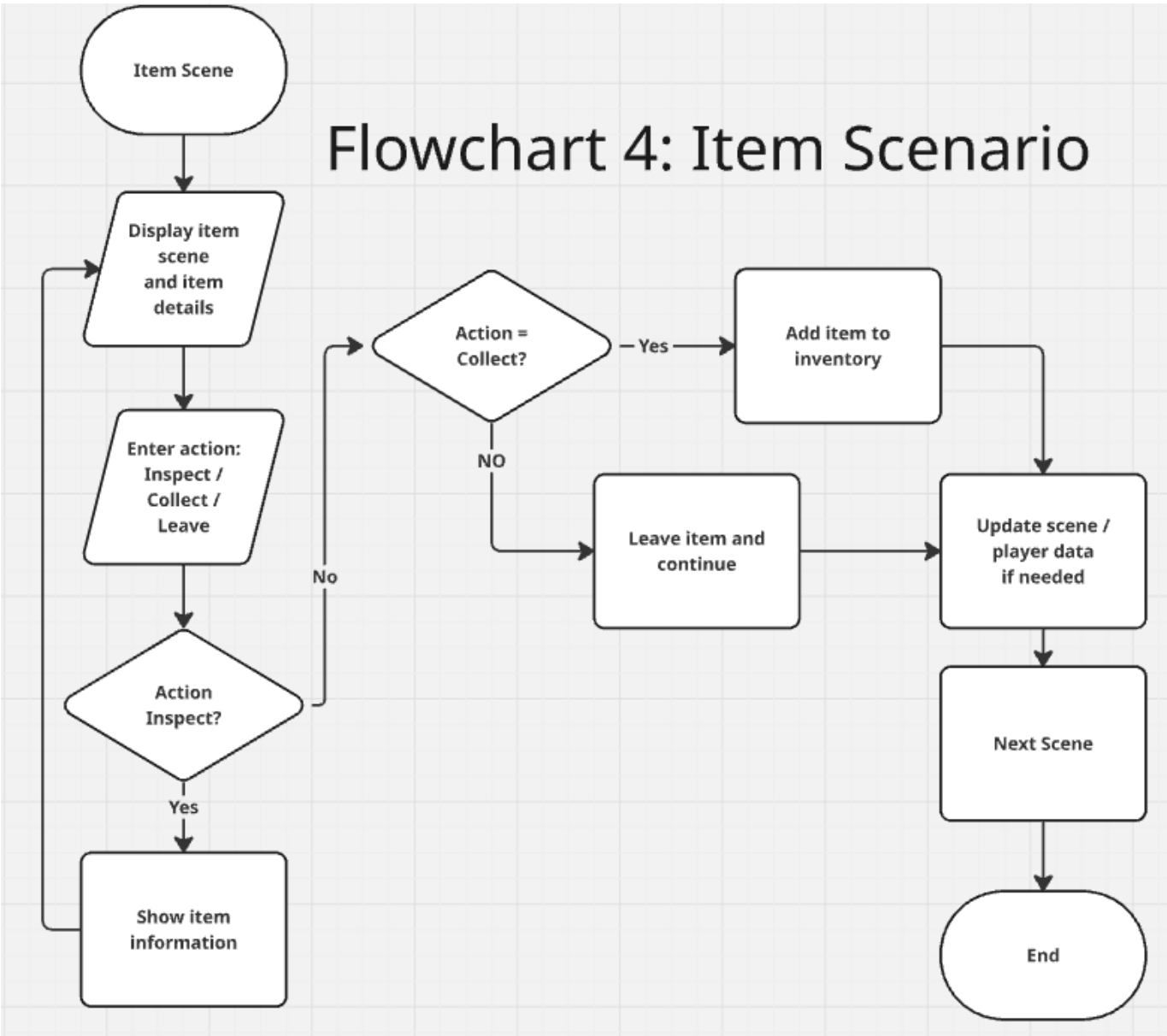
Feature	Inputs	Processing	Outputs
Use item	Item selection (item number)	Validate selection, apply item effect (heal/buff/unlock), remove item if consumable.	Confirmation message + updated stats/route access.
Save game	Save choice (Y/N), filename/slot (string)	Validate filename/slot, write game state to file, handle errors safely.	Save success message OR save error message.

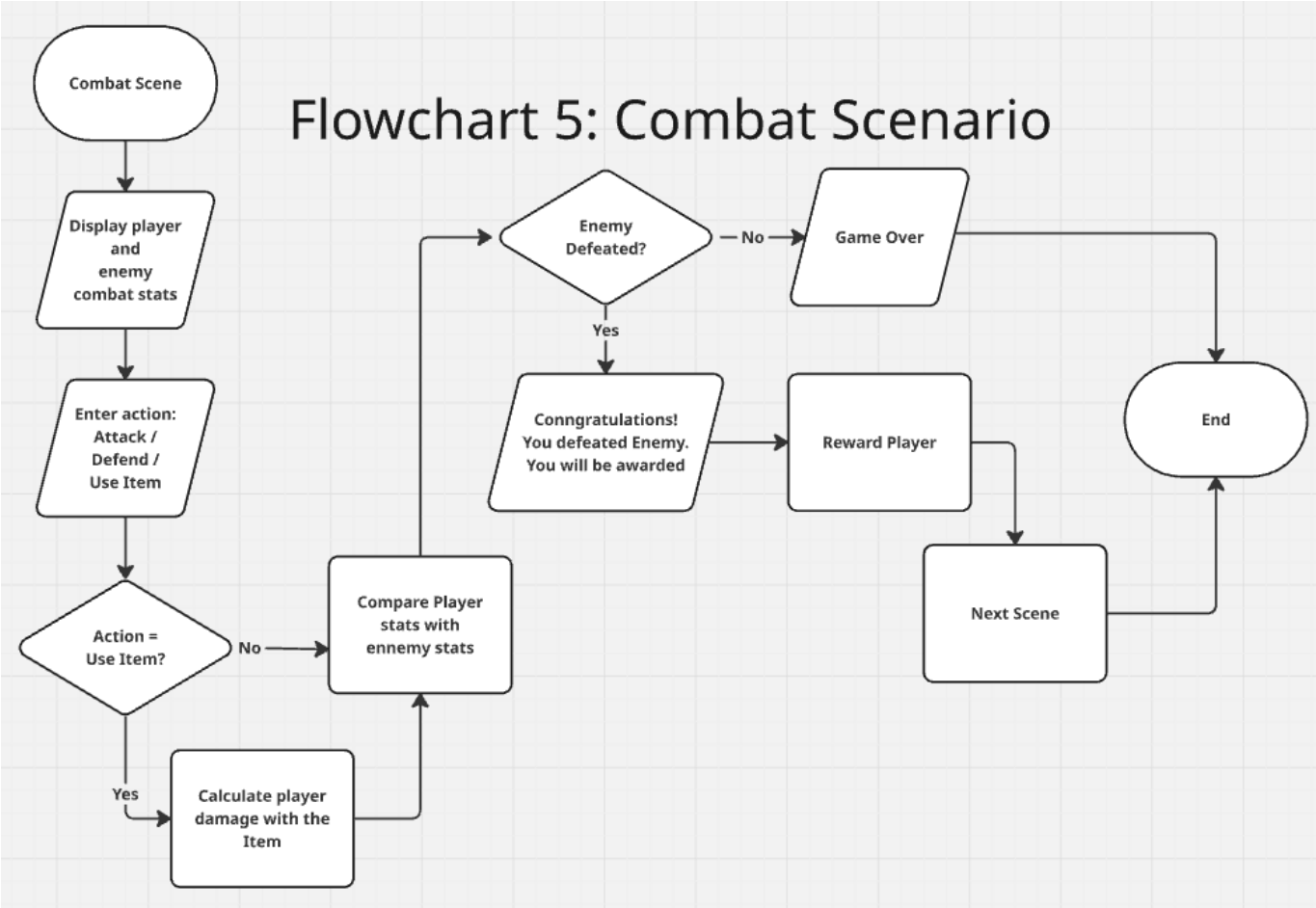
Algorithm:

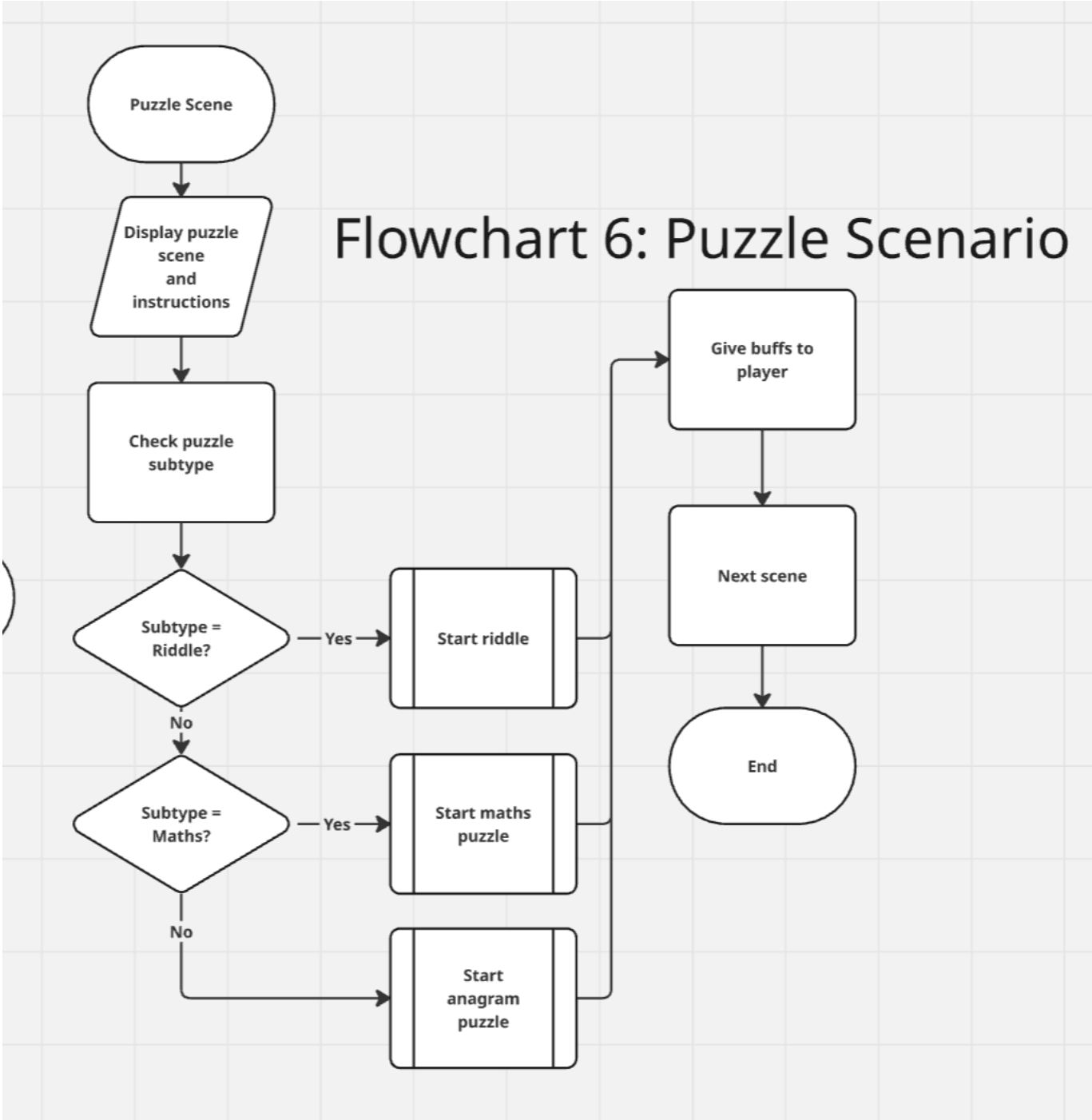


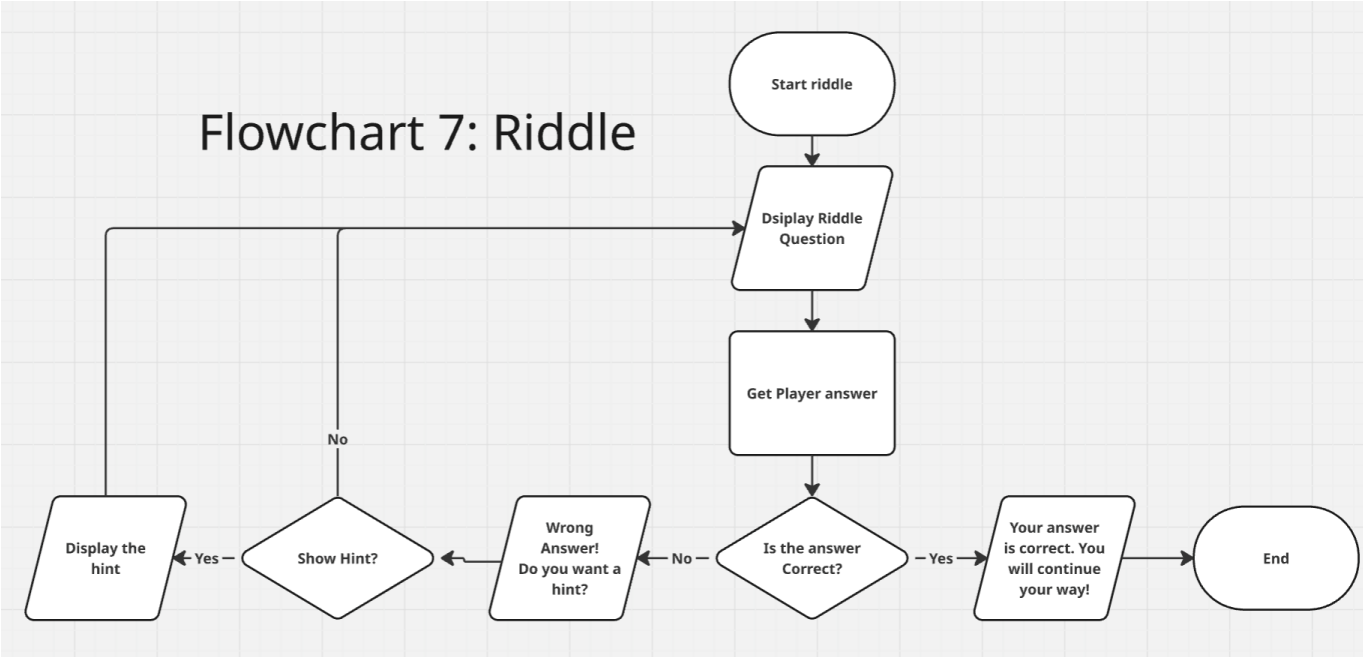

Flowchart 2: Start New Game
New Game
Create new player session
Set default health, stats, and inventory.
Set current scene = 1
Display game introduction
Load first scene
Scene type = Item?
Yes
Item Scenario
No
Scene type = Combat?
Yes
Combat Scenario
No
Puzzle Scenario
Next Scene
End

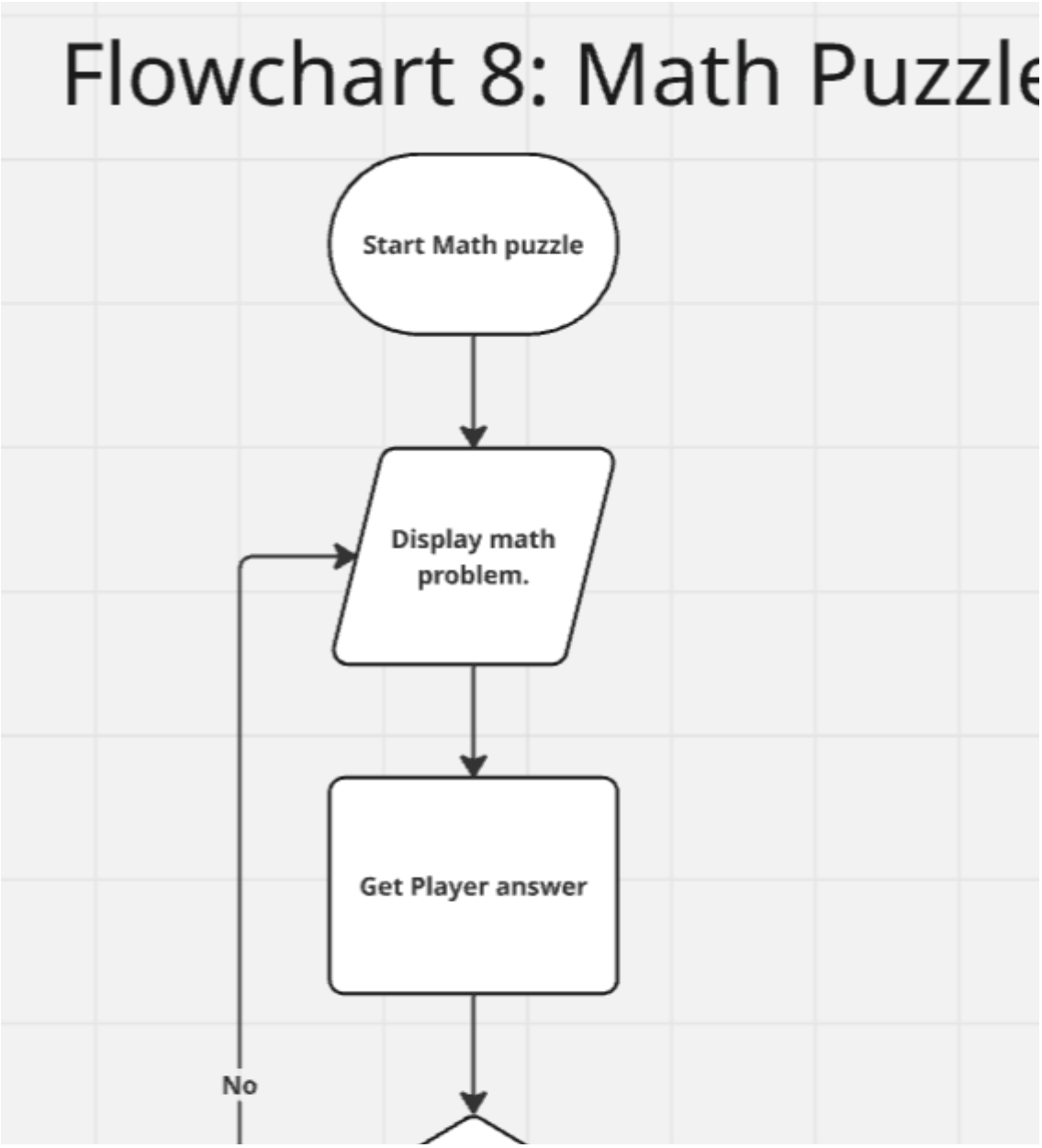

Flowchart 3: Load Game
Load Game
Enter save slot or save file
Save found?
Yes
No
Show error and ask again
Load saved player data and stats
Load saved scene
Game was loaded successfully. Enjoy your game!
End

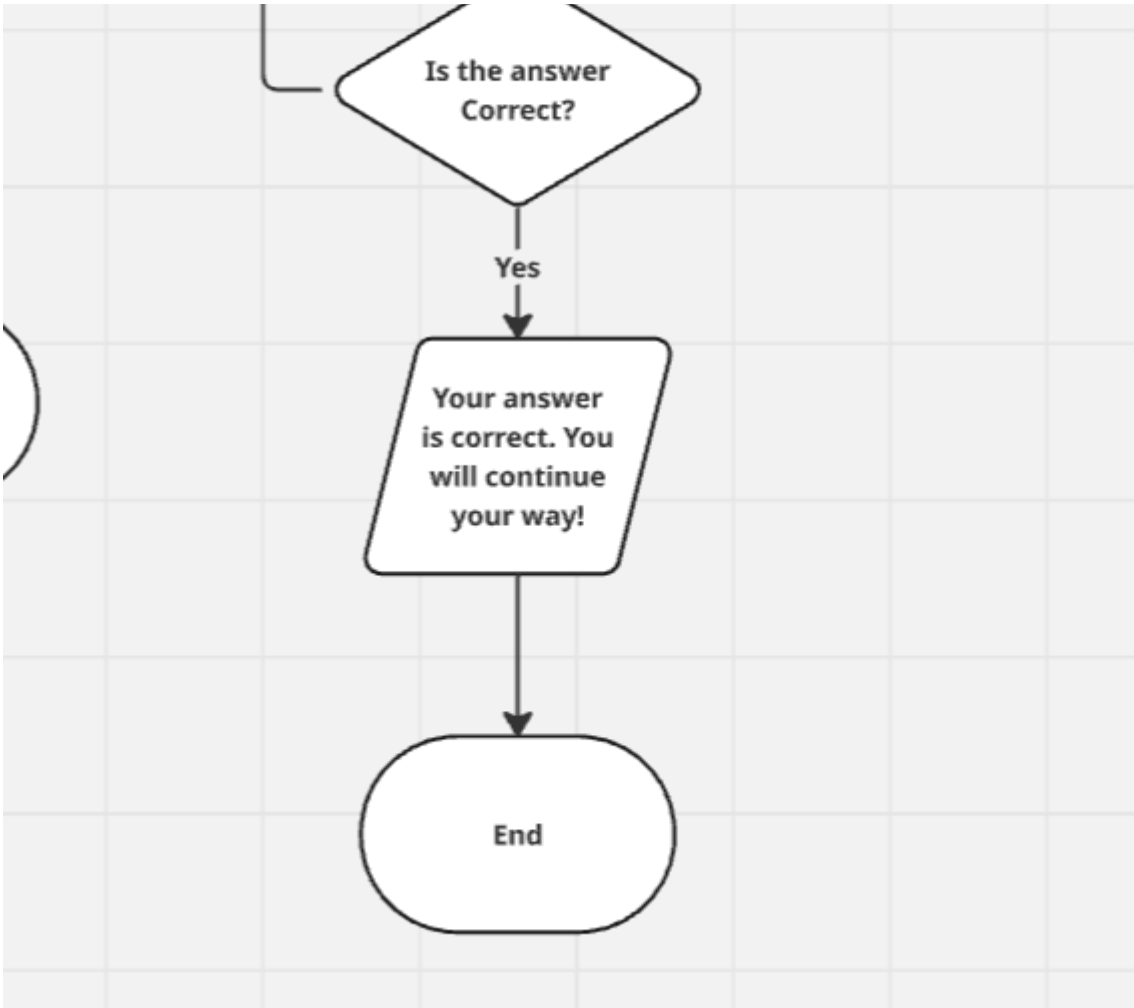

Flowchart 4: Item Scenario
Item Scene
Display item scene and item details
Enter action: Inspect / Collect / Leave
Action Inspect?
Yes
Show item information
No
Action = Collect?
Yes
Add item to inventory
NO
Leave item and continue
Update scene / player data if needed
Next Scene
End

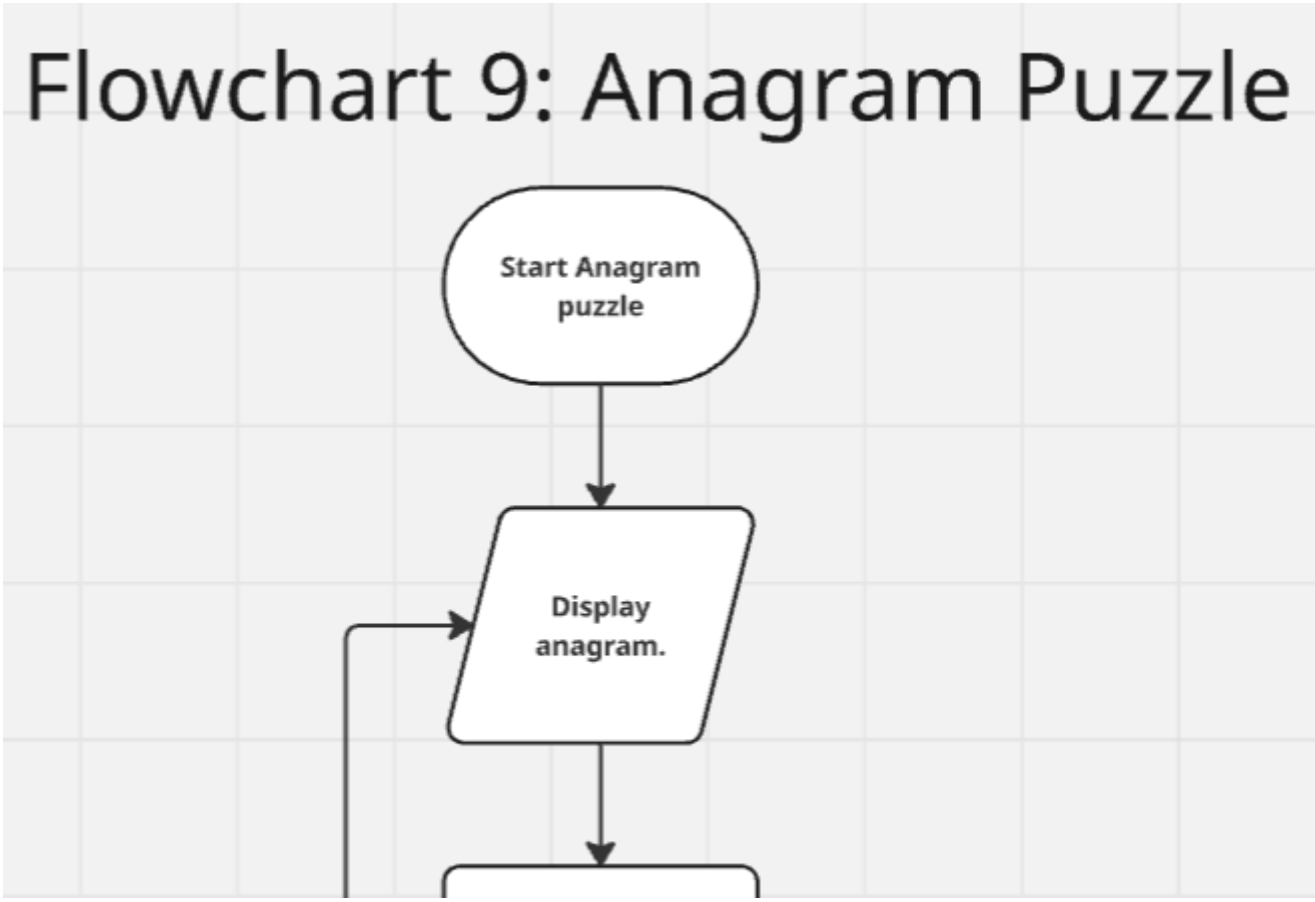

Flowchart 5: Combat Scenario
Combat Scene
Display player and enemy combat stats
Enter action: Attack / Defend / Use Item
Action = Use Item?
No
Yes
Calculate player damage with the Item
Compare Player stats with ennemy stats
Enemy Defeated?
No
Game Over
Yes
Conngratulations! You defeated Enemy. You will be awarded
Reward Player
Next Scene
End

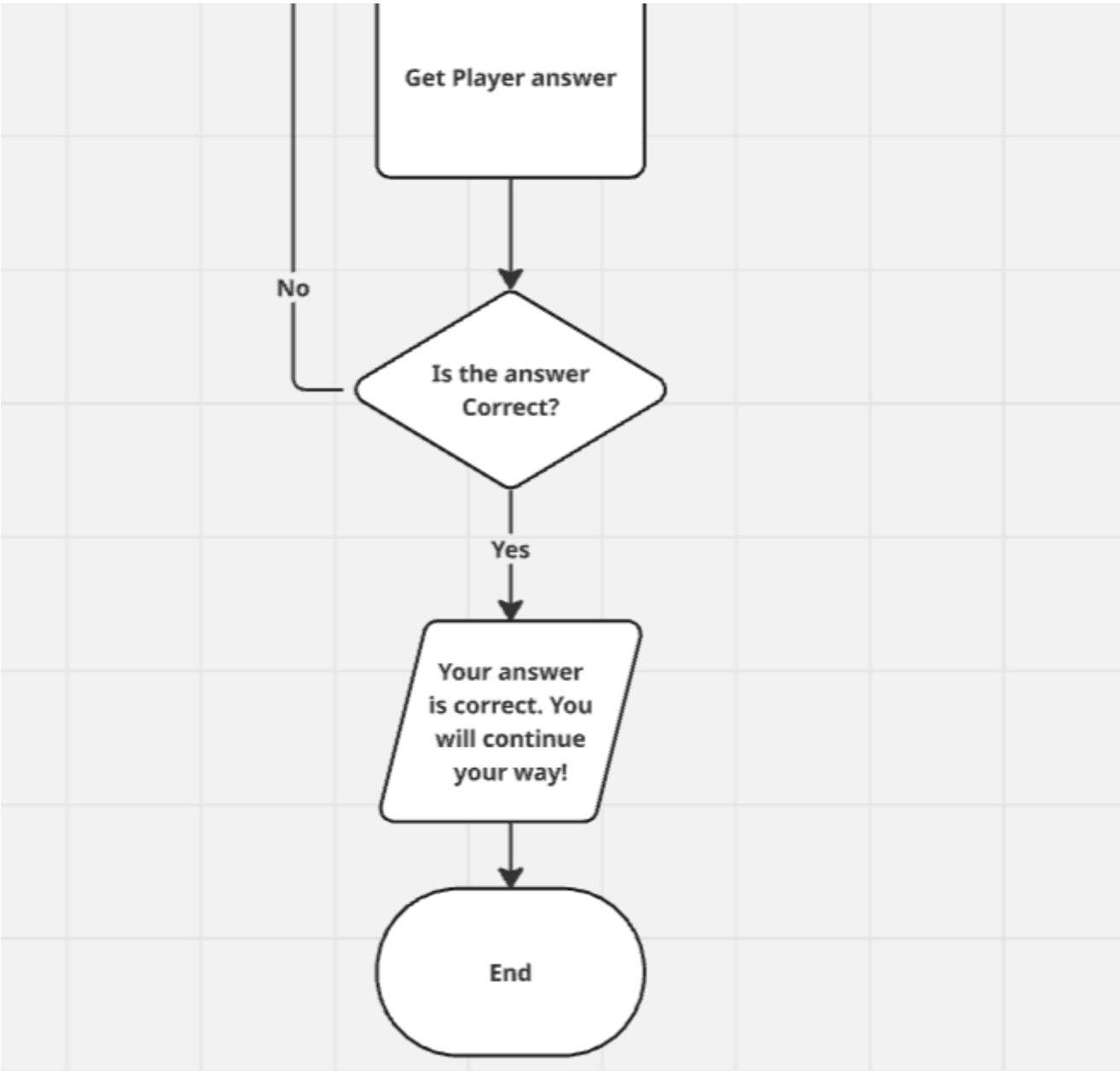

Flowchart 6: Puzzle Scenario
Puzzle Scene
Display puzzle scene and instructions
Check puzzle subtype
Subtype = Riddle?
Yes
No
Start riddle
Subtype = Maths?
Yes
No
Start maths puzzle
Start anagram puzzle
Give buffs to player
Next scene
End


Flowchart 7: Riddle
Start riddle
Dsiplay Riddle Question
Get Player answer
Is the answer Correct?
Yes
Your answer is correct. You will continue your way!
End
No
Wrong Answer! Do you want a hint?
Show Hint?
Yes
Display the hint
No


Flowchart 8: Math Puzzle
Start Math puzzle
Display math problem.
Get Player answer
No


Is the answer Correct?
Yes
Your answer is correct. You will continue your way!
End


Flowchart 9: Anagram Puzzle
Start Anagram puzzle
Display anagram.



Class Diagram:

In progress...

Testing:

I am willin to do the tests once I finish the code completely.

Test Table:

Test No:	Item to test	Test No. Item to test Test Description (with example data)	Expected Result	Actual Result	Comments / Actions

Annotation Screenshots:

Technical elements:

References:

Appendix Code Listing:

```
#include <iostream>
#include <vector>
#include <string>
#include <random>
#include <limits>
#include <fstream>

using namespace std;

class Player {
private:
    string name_;
    int health_;
    int lives_;
    int attack_power_;
    int defense_;
    int score_;
    vector<string> inventory_;

public:
    Player() {
        name_ = "Player";
        health_ = 10;
        lives_ = 3;
        attack_power_ = 4;
        defense_ = 0;
        score_ = 0;
    }

    void SetName(const string& new_name) {
        name_ = new_name;
    }

    string GetName() const {
        return name_;
    }

    int GetHealth() const {
        return health_;
    }

    int GetLives() const {
        return lives_;
    }

    int GetAttackPower() const {
        return attack_power_;
    }

    int GetDefense() const {
```

```
        return defense_;
    }

    int GetScore() const {
        return score_;
    }

    void AddScore(int amount) {
        score_ += amount;
    }

    void ChangeHealth(int amount) {
        health_ += amount;

        if (health_ < 0) {
            health_ = 0;
        }
    }

    void ChangeAttackPower(int amount) {
        attack_power_ += amount;

        if (attack_power_ < 0) {
            attack_power_ = 0;
        }
    }

    void ChangeDefense(int amount) {
        defense_ += amount;

        if (defense_ < 0) {
            defense_ = 0;
        }
    }

    void LoseLife() {
        if (lives_ > 0) {
            lives_--;
        }
    }

    void ResetForNewLife() {
        health_ = 10;
    }

    void ResetAll() {
        name_ = "Player";
        health_ = 10;
        lives_ = 3;
        attack_power_ = 4;
        defense_ = 0;
```

```
        score_ = 0;
        inventory_.clear();
    }

    void AddItem(const string& item_name, int hp_delta, int atk_delta, int def_delta,
int score_delta) {
        inventory_.push_back(item_name);
        health_ += hp_delta;
        attack_power_ += atk_delta;
        defense_ += def_delta;
        score_ += score_delta;

        cout << "You obtained: " << item_name << "\n";

        if (hp_delta != 0) {
            cout << "HP " << (hp_delta > 0 ? "+" : "") << hp_delta << "\n";
        }

        if (atk_delta != 0) {
            cout << "ATK " << (atk_delta > 0 ? "+" : "") << atk_delta << "\n";
        }

        if (def_delta != 0) {
            cout << "DEF " << (def_delta > 0 ? "+" : "") << def_delta << "\n";
        }

        if (score_delta != 0) {
            cout << "Score " << (score_delta > 0 ? "+" : "") << score_delta << "\n";
        }
    }

    bool HasItem(const string& item_name) const {
        for (const string& item : inventory_) {
            if (item == item_name) {
                return true;
            }
        }

        return false;
    }

    void ShowInventory() const {
        cout << "\n--- Inventory ---\n";

        if (inventory_.empty()) {
            cout << "Inventory is empty.\n";
        }
        else {
            for (const string& item : inventory_) {
                cout << "- " << item << "\n";
            }
```

```
        }
    }

    void ShowStats() const {
        cout << "\n--- Player Stats ---\n";
        cout << "Name: " << name_ << "\n";
        cout << "Health: " << health_ << "\n";
        cout << "Lives: " << lives_ << "\n";
        cout << "Attack Power: " << attack_power_ << "\n";
        cout << "Defense: " << defense_ << "\n";
        cout << "Score: " << score_ << "\n";
    }
};

struct StorySceneData {
    int scene_id;
    string description;
    string choice_1;
    string choice_2;
    int next_scene_1;
    int next_scene_2;
};

class SceneLoader {
public:
    vector<string> SplitCsvLine(const string& line) {
        vector<string> fields;
        string current_field;
        bool inside_quotes = false;

        for (char current_char : line) {
            if (current_char == '"') {
                inside_quotes = !inside_quotes;
            }
            else if (current_char == ',' && !inside_quotes) {
                fields.push_back(current_field);
                current_field.clear();
            }
            else {
                current_field += current_char;
            }
        }

        fields.push_back(current_field);
        return fields;
    }

    vector<StorySceneData> LoadStoryScenes(const string& file_name) {
        vector<StorySceneData> story_scenes;
        ifstream file(file_name);
        string line;
```

```
        if (!file.is_open()) {
            cout << "Error: could not open " << file_name << "\n";
            return story_scenes;
        }

        getline(file, line);

        while (getline(file, line)) {
            vector<string> fields = SplitCsvLine(line);

            if (fields.size() == 6) {
                StorySceneData scene;
                scene.scene_id = stoi(fields[0]);
                scene.description = fields[1];
                scene.choice_1 = fields[2];
                scene.choice_2 = fields[3];
                scene.next_scene_1 = stoi(fields[4]);
                scene.next_scene_2 = stoi(fields[5]);

                story_scenes.push_back(scene);
            }
        }

        file.close();
        return story_scenes;
    }

    StorySceneData GetStorySceneById(const vector<StorySceneData>& story_scenes, int
scene_id) {
        StorySceneData empty_scene;
        empty_scene.scene_id = -1;

        for (const StorySceneData& scene : story_scenes) {
            if (scene.scene_id == scene_id) {
                return scene;
            }
        }

        return empty_scene;
    }
};

class Game {
private:
    Player player_;
    vector<StorySceneData> story_scenes_;
    SceneLoader scene_loader_;

public:
    void StartGame() {
```

```
        cin.ignore(numeric_limits<streamsize>::max(), '\n');

        string player_name;
        cout << "Enter your name: ";
        getline(cin, player_name);

        player_.SetName(player_name);

        cout << "\nWelcome to Ravenspire Castle, " << player_.GetName() << "!\n";
        player_.ShowStats();

        story_scenes_ = scene_loader_.LoadStoryScenes("scenes.csv");
        StorySceneData first_scene = scene_loader_.GetStorySceneById(story_scenes_,
1);

        if (first_scene.scene_id == -1) {
            cout << "Error: first scene not found.\n";
        }
        else {
            cout << "\n" << first_scene.description << "\n";
            cout << "1. " << first_scene.choice_1 << "\n";
            cout << "2. " << first_scene.choice_2 << "\n";
        }
    }

    void LoadGame() {
        cout << "Load Game selected.\n";
    }
};

class Scene {
};

class PuzzleScene {
public:
};

class CombatScene {
};

class ItemScene {
};

int main() {
    int choice;
    Game game;

    do {
        cout << "\n--- RAVENSPIRE CASTLE ---\n";
        cout << "1. Start Game\n";
        cout << "2. Load Game\n";
```

```
        cout << "3. Exit\n";
        cout << "Enter your choice: ";
        cin >> choice;

        if (cin.fail()) {
            cin.clear();
            cin.ignore(numeric_limits<streamsize>::max(), '\n');
            choice = 0;
            cout << "Invalid input. Please enter a number.\n";
        }
        else if (choice < 1 || choice > 3) {
            cout << "Invalid choice. Please enter 1, 2, or 3.\n";
        }

    } while (choice < 1 || choice > 3);

    switch (choice) {
        case 1:
            cout << "Start Game selected.\n";
            game.StartGame();
            break;

        case 2:
            game.LoadGame();
            break;

        case 3:
            cout << "Exiting game...\n";
            break;
    }

    return 0;
}
```